

Speech and language technology (DA 3452)

...

Lecture 3 – Aug 4
Text Normalization, Byte Pair Encoding (BPE)

Module 1

Representing Language Computationally

Text Normalization to Subword Tokens

Recap — ASCII

- **ASCII – American Standard Code for Information Interchange**
 - Developed for English-language computing
 - Its assignments are retained in Unicode’s Basic Latin block
- **ASCII is a 7-bit encoding (commonly store in one 8-bit byte)**
 - 128 possible codes

Ch	Hex	Dec	Ch	Hex	Dec	Ch	Hex	Dec	Ch	Hex	Dec	
<	3C	60	@	40	64	...	\	5C	92	`	60	96
=	3D	61	A	41	65	...	[	5D	93	a	61	97
>	3E	62	B	42	66	...	^	5E	94	b	62	98
?	3F	63	C	43	67	...	_	5F	95	c	63	99

Figure 2.4 Some selected ASCII codes for some English letters, with the codes shown both in hexadecimal and decimal.

But ASCII cannot represent é, தமிழ், हिंदी, 中, or 😊. How can one system represent the world’s writing systems?

Recap – Unicode Standard

- Unicode standard

- Represents characters used in the world's writing systems
 - Includes punctuation, mathematical symbols, currency symbols, and emojis
- Unicode 17.0 has 159,801 characters

- Each encoded character is assigned a Code Point

- Code Point - Numerical identifier for an abstract character
- Written in Hexadecimal. From 0 to 0x10FFFF (which is 1,114,111 decimal)
- It is traditional to represent these code points with the prefix “U+” (U+0000 to U+10FFFF)

U+0061	a	LATIN SMALL LETTER A
U+0062	b	LATIN SMALL LETTER B
U+0063	c	LATIN SMALL LETTER C
U+00F9	ù	LATIN SMALL LETTER U WITH GRAVE
U+00FA	ú	LATIN SMALL LETTER U WITH ACUTE
U+00FB	û	LATIN SMALL LETTER U WITH CIRCUMFLEX
U+00FC	ü	LATIN SMALL LETTER U WITH DIAERESIS
U+8FDB	进	
U+8FDC	远	
U+8FDD	违	
U+8FDE	连	
U+1F600	😄	GRINNING FACE
U+1F00E	八萬	MAHJONG TILE EIGHT OF CHARACTERS

Abstract character ≠ visual representation (glyph)



Next : How do we store text using code points?

Recap – Direct storage

Unicode has assigned each character a number. Why not simply store that number?

Characters	h	e	l	l	o
Unicode code points	U+0068	U+0065	U+006C	U+006C	U+006F

If every code point is stored in 4 bytes:

00 00 00 68

00 00 00 65

00 00 00 6C

00 00 00 6C

00 00 00 6F

5 characters -> 20 bytes

UTF-32

- Simple and uniform
- Works for all Unicode code points
- But wasteful for ordinary English text

Can we use WhatsApp this way? Can we be more efficient?

Recap : UTF-8 (Variable Length Encoding)

- Fixed-width encoding (UTF-32) is wasteful and inefficient for large systems
 - Also zeros cause backward compatibility issues with legacy software
- UTF-8 : Most common encoding standard
 - Unicode Transformation Format - 8 bits
 - Variable length encoding
 - ASCII characters: 1 byte. Higher Unicode ranges: 2–4 bytes
 - ASCII byte sequence is same in ASCII and UTF-8

UTF-32: 00 00 00 68 | 00 00 00 65 | 00 00 00 6C | 00 00 00 6C | 00 00 00 6F : 20 bytes

UTF-8: 68 | 65 | 6C | 6C | 6F : 5 bytes

Code Points		UTF-8 Encoding			
From - To	Bit Value	Byte 1	Byte 2	Byte 3	Byte 4
U+0000-U+007F	0xxxxxxx	xxxxxxx			
U+0080-U+07FF	0000yyy yyxxxxxx	110yyyy	10xxxxx		
U+0800-U+FFFF	zzzzyyyy yyxxxxxx	1110zzzz	10yyyyy	10xxxxx	
U+010000-U+10FFFF	000uuuuu zzzzyyyy yyxxxxxx	11110uuu	10uuzzzz	10yyyyy	10xxxxx

Figure 2.5 Mapping from Unicode code point to the variable length UTF-8 encoding. For a given code point in the From-To range, the bit value in column 2 is packed into 1, 2, 3, or 4 bytes. Figure adapted from Unicode 16.0 Core Spec Chapter 3 Table 3-6.

Module 1: Where we are

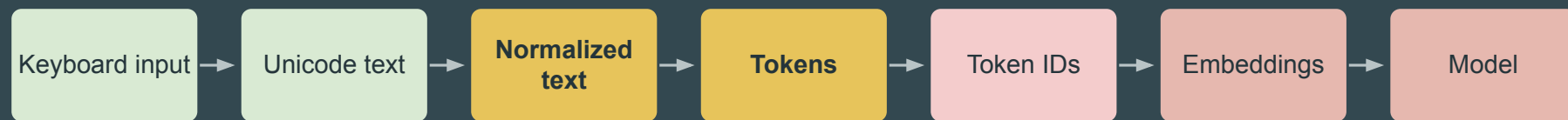
1 The Text Representation Problem

2 Characters and Unicode

3 **Normalized Text → Subword Tokens**

4 Tokens → Document Vectors

Recall : From Keyboard to Model



Normalization

Two identical-looking strings

café

café

Are these the same string?

Canonical equivalence

Precomposed

é

U+00E9



Decomposed

e + é

U+0065 + U+0301

Different valid **code-point sequences** can represent the same abstract text.

Same appearance can be generated by different code points

STRING A

c a f é

U+0063 U+0061 U+0066 U+00E9

STRING B

c a f e ó

U+0063 U+0061 U+0066 U+0065 U+0301

Visible text ≠ code-point sequence ≠ computational identity

The uniqueness problem – 1

The same visual appearance can be generated by two different code-point sequences

café

café

String A

String B

```
bvs@Balajis-MacBook-Pro ~ % printf 'caf\u00E9\n'  
café  
bvs@Balajis-MacBook-Pro ~ % printf 'cafe\u0301\n'  
café
```

é

U+00E9

e + ́

U+0065 + U+0301

String A

UTF-8 : C3 A9

String B

UTF-8 : 65 CC 81

How can we make canonically equivalent Unicode strings consistent?

Canonical Equivalence vs Compatibility Equivalence

Canonical Equivalence – Same abstract text



Compatibility Equivalence – Visually different but same “function”

Subtype	Examples
Font variants	§ → H
	HH → H
Linebreaking differences	[NBSP] → [SPACE]
Positional variant forms	ع → ع
	ع → ع
	ع → ع
	ع → ع
Circled variants	① → 1
Width variants	カ → カ
Rotated variants	{ → {
	} → }
Superscripts/subscripts	i ⁹ → i ₉
	i ₉ → i ⁹
Squared characters	アパート → アパート
Fractions	¼ → 1/4
Other	dž → dž

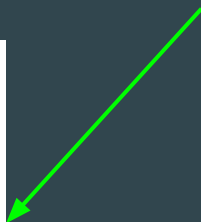


Figure 1. Examples of Canonical Equivalence

Subtype	Examples
Combining sequence	Ç ↔ C+◌̇
Ordering of combining marks	q+◌̇+◌̇ ↔ q+◌̇+◌̇
Hangul & conjoining jamo	가 ↔ ㄱ + ㅏ
Singleton equivalence	Ω ↔ Ω

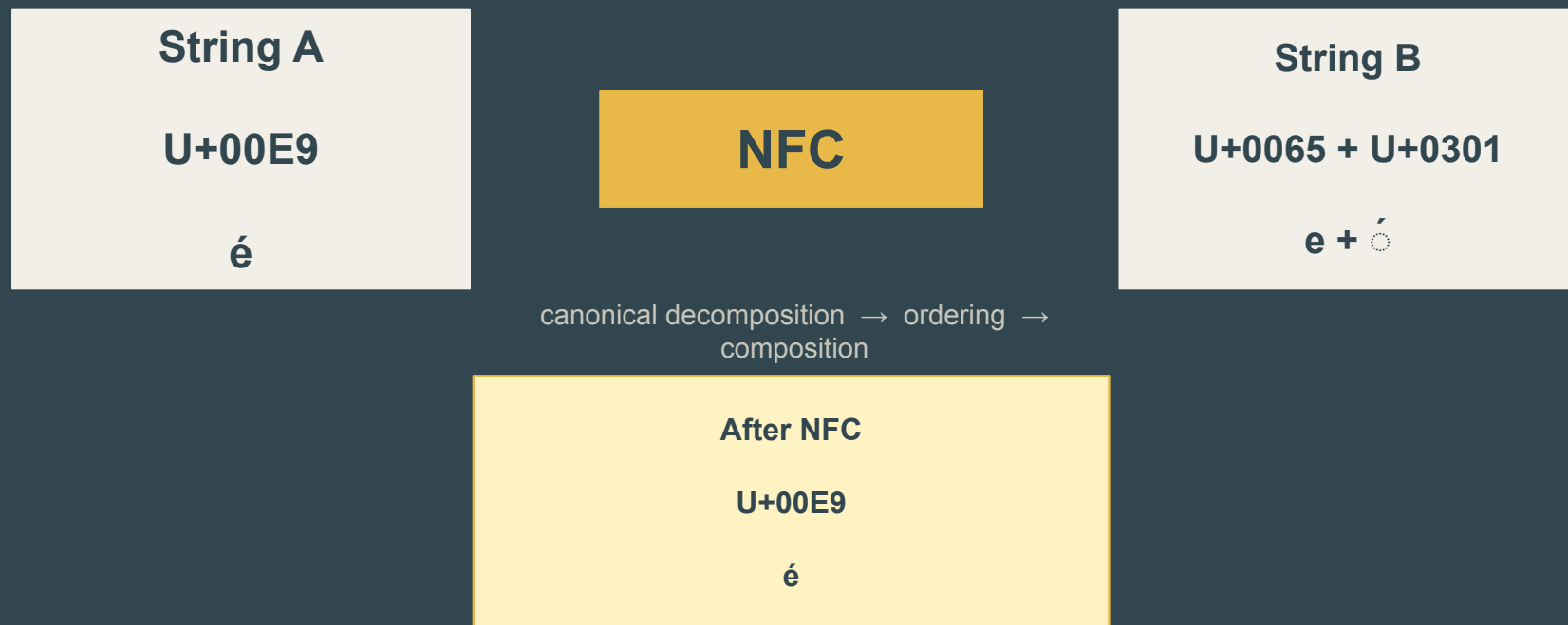
Form	Description
Normalization Form D (NFD)	Canonical Decomposition
Normalization Form C (NFC)	Canonical Decomposition, followed by Canonical Composition
Normalization Form KD (NFKD)	Compatibility Decomposition
Normalization Form KC (NFKC)	Compatibility Decomposition, followed by Canonical Composition

Unicode normalization: four standard forms

	Decomposed	Composed where possible
Canonical equivalence	NFD Canonical decomposition $\acute{e} \rightarrow e + \textcircled{\cdot}$	NFC Canonical decomposition, then composition $e + \textcircled{\cdot} \rightarrow \acute{e}$
Compatibility equivalence	NFKD Compatibility decomposition $fi \rightarrow f + i \quad \textcircled{1} \rightarrow 1$	NFKC Compatibility decomposition, then composition $fi \rightarrow fi \quad \textcircled{1} \rightarrow 1$

For this course, we will mostly use NFC. You need not memorize any of the algorithms

NFC: choose a consistent canonical representation



NFC makes canonically equivalent strings compare the same, count the same, and tokenize the same.

NFC normalization

Before NFC

café ≠ café



After NFC

café = café

```
unicodedata.normalize("NFC", s)
```

Normalize before comparing, indexing, or tokenizing.

NFC normalization in Python

Code

```
import unicodedata

a = "caf\u00E9"      # precomposed
b = "cafe\u0301"     # decomposed

print(a == b)
print(len(a), len(b))

a = unicodedata.normalize("NFC", a)
b = unicodedata.normalize("NFC", b)

print(a == b)
print([f"U+{ord(c):04X}" for c in b])
```

Output

```
False
4 5
True
['U+0063', 'U+0061',
 'U+0066', 'U+00E9']
```

What changed?

- The displayed string did not change.
- Equality changed: before NFC → False; after NFC → True.
- The representation is now stable for counting and tokenization.

A second normalization problem

When should we distinguish in case, punctuation, disfluencies, etc?

Apple → apple

Is capitalization irrelevant?

I'm → I am

Are these interchangeable for the task?

₹1,000 → 1000

What information has been removed?

um, the model... → the model...

Is the filler noise or evidence?

We need Task Normalization

Two meanings of normalization

UNICODE NORMALIZATION

Standardize equivalent representations

é ↔ e + é

Preserves the intended abstract text

TASK-SPECIFIC NLP NORMALIZATION

Choose which distinctions to remove

Apple → apple

I'm → I am

₹1,000 → 1000

um → [removed]

Task normalization is a modelling decision: what information are we willing to discard?

Tokenization

Normalized text is still not **tokens**

Even after normalization, what should the model units be?

Words

Characters

Morphemes

Learned subwords

Why not whole words?

cat cats catlike catlikeish
DA3452 names spelling variants

- uncertain boundaries
- huge and growing vocabulary
- unseen words never disappear

Vocabulary



keeps growing

Words are meaningful—but the vocabulary never stops growing.

Why not only characters?

[i][n][t][e][r][n][a][t][i][o][n][a][l][i][z][a][t][i][o][n]

20 character tokens

Advantages

Full coverage

Every word can be represented using basic characters.

Compact vocabulary

The tokenizer needs only letters, symbols, and other basic units.

Main cost

Long sequences of small units

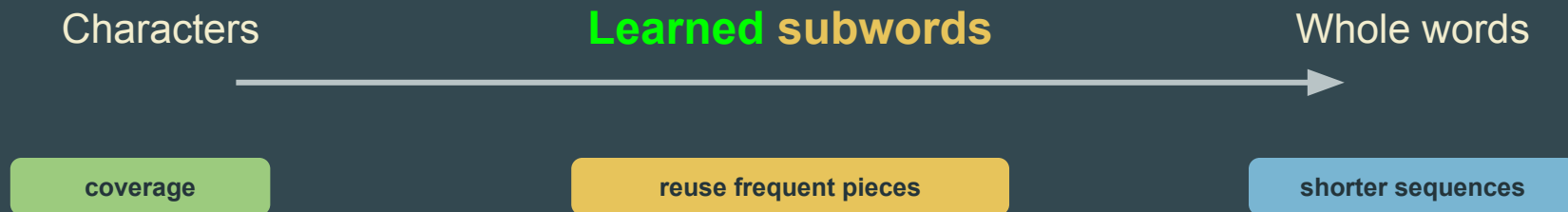
- More token positions must be processed.
- Frequent patterns must be reconstructed repeatedly.

inter nation al ization

[inter][nation][al][ization]

Illustrative subword segmentation: 4 tokens
Actual segmentation depends on the learned tokenizer.

Learned subwords



low lower lowest → low + er | low + est

Tokenization keeps frequent strings intact; breaks rare strings into **reusable pieces**.

Training a tokenizer versus using it

TRAINER (done once)

Training corpus + initial symbols +
vocabulary budget



Tokenizer trainer



Learned tokenizer
vocabulary + ordered merge rules

Example learned from corpus

Vocabulary additions: `ne`, `new`, `re`, ...
Merge order: 1. `n+e→ne` 2. `ne+w→new` 3.
`r+e→re`

The vocabulary and rules are learned here.

vs

ENCODE (each input)

New normalized text + fixed learned
tokenizer



Token strings



Token IDs

Example encoding

`newest` → `[new]` `[e]` `[s]` `[t]`

`[417]` `[68]` `[84]` `[85]` illustrative IDs

No new tokens or rules are learned here.

Training learns the tokenizer. Encoding applies the fixed tokenizer to new text.

An unseen word is represented using existing tokens; it is not automatically added to the vocabulary.

Byte-Pair Encoding (BPE): the core idea



Let us take a “corpus” – **set, new, new, renew, reset, renew**

In practice, we may start with “bytes” instead of characters. In the example, we will use characters

BPE training example: initial corpus

Corpus: set new new renew reset renew

new

x2

n e w

renew

x2

r e n e w

set

x1

s e t

reset

x1

r e s e t

corpus

2 _ n e w

2 _ r e n e w

1 s e t

1 _ r e s e t

vocabulary

_ , e , n , r , s , t , w

Question: which adjacent pair is most frequent? Notice that there is a tie (which we break with some preset rules)

BPE merge 1: $n + e \rightarrow ne$

Updated corpus

new ×2 **ne** w

renew ×2 r e **ne** w

set ×1 s e t

reset ×1 r e s e t

vocabulary

_, e, n, r, s, t, w, ne

Selected pair

$n + e \rightarrow ne$
Pair frequency = 4

new ×2 → 2 occurrences
renew ×2 → 2 occurrences
Total → **4**

Tokenizer state

Vocabulary
added:

ne

Merge rules:

1. $n + e \rightarrow ne$

The corpus is updated, ne is added to the vocabulary, and the merge rule is recorded.

BPE merge 2: ne + w → new

Updated corpus

new ×2

new

renew ×2

r e new

set ×1

s e t

reset ×1

r e s e t

vocabulary

_, e, n, r, s, t, w, ne, new

Selected pair

ne + w → new

Pair frequency in corpus = 4

new ×2 → 2 occurrences

renew ×2 → 2 occurrences

Total → 4

Tokenizer state

Vocabulary
added:
new

Merge rules:

1. n + e → ne
2. ne + w → new

The frequent sequence new becomes a reusable token, and the merge list is updated.

BPE merge 3: $r + e \rightarrow re$

Updated corpus

new ×2	new
renew ×2	re new
set ×1	s e t
reset ×1	re s e t

Vocabulary growth

characters

+ ne

+ new

+ re

Selected pair

$r + e \rightarrow re$

Pair frequency in corpus = 3

renew ×2 → 2 occurrences
reset ×1 → 1 occurrences
Total → 3

Tokenizer state

Vocabulary
added:
re

Merge rules:

1. $n + e \rightarrow ne$
2. $ne + w \rightarrow new$
3. $r + e \rightarrow re$

The frequent sequence **re** becomes a reusable token, and the merge list is updated.

Continuing the BPE training example

Step	Selected pair	New token	Added to V
1	n + e	ne	{ne}
2	ne + w	new	{new}
3	r + e	re	{re}
4	s + e	se	{se}
5	se + t	set	{set}

Ordered merge rules

1. n + e → ne
2. ne + w → new
3. r + e → re
4. s + e → se
5. se + t → set

Tokenizer state after 5 merges

Initial symbols: {e, n, r, s, t, w}

Added tokens: {ne, new, re, se, set}

Vocabulary = initial symbols + added tokens

Each iteration updates the corpus, adds one vocabulary item, and records one ordered merge rule.

The BPE Algorithm



function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters
for $i = 1$ **to** k **do** # merge tokens k times
 $t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C
 $t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating
 $V \leftarrow V + t_{NEW}$ # update the vocabulary
 Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus
return V

Encoding an unseen word

newest

Start

n e w e s t

Apply $n+e \rightarrow ne$

ne w e s t

Apply $ne+w \rightarrow new$

new e s t

Final tokens

[new] [e] [s] [t]

The whole word was **unseen**—but it can still be encoded.